G.H. Raisoni College of Engineering and Management
Department of Information Technology

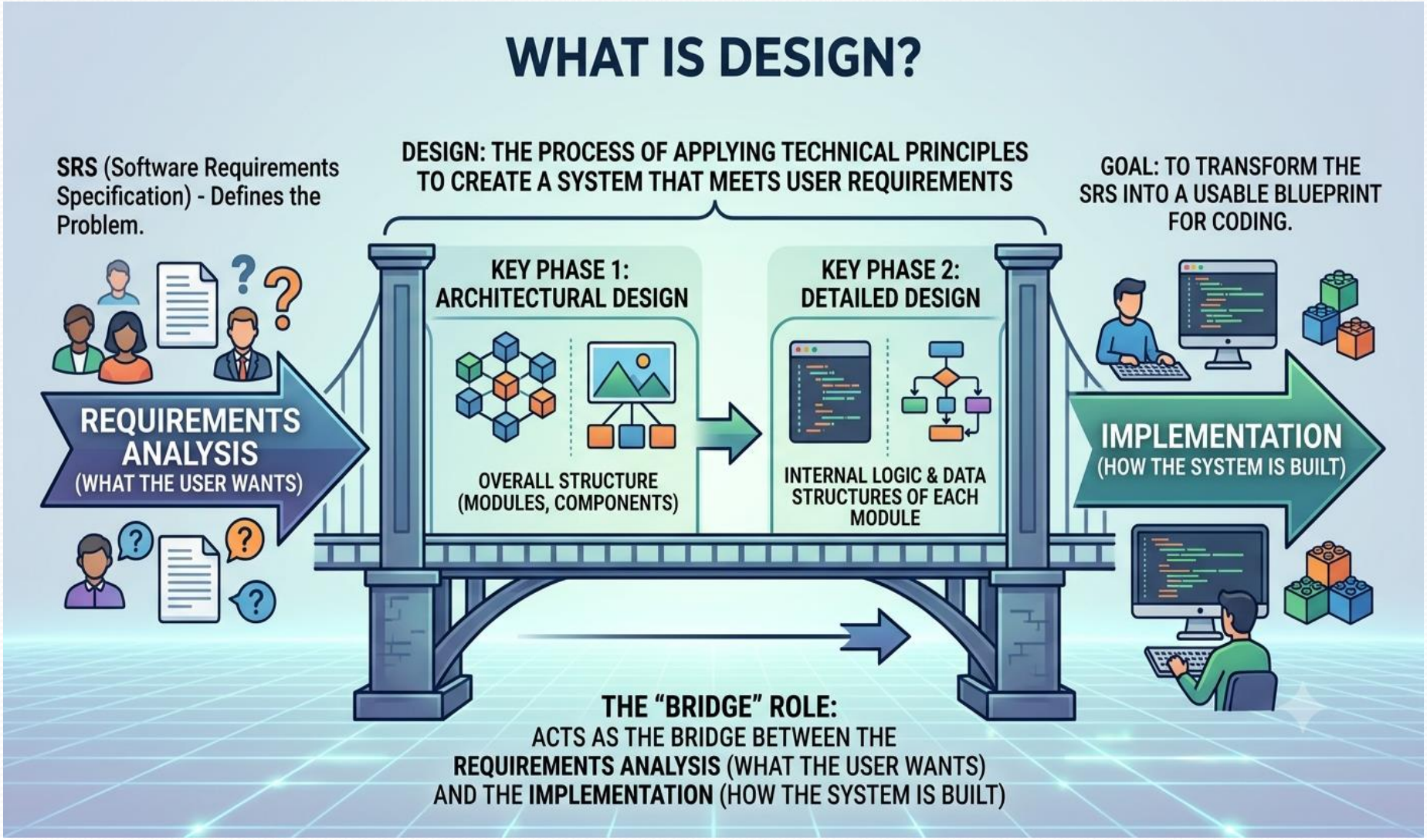
Unit 3 : SOFTWARE DESIGN

Prof. Sagar Mahajan
Prof. Shivkumar Lilhare

Overview of Design Process

- **What is Design?** The process of applying technical principles to create a system that meets user requirements.
- **The "Bridge" Role:** It acts as the bridge between the *Requirements Analysis* (What the user wants) and the *Implementation* (How the system is built).
- **Key Phases:**
  - **Architectural Design:** Defining the overall structure (modules, components).
  - **Detailed Design:** Defining the internal logic and data structures of each module.
- **Goal:** To transform the SRS (Software Requirements Specification) into a usable blueprint for coding.

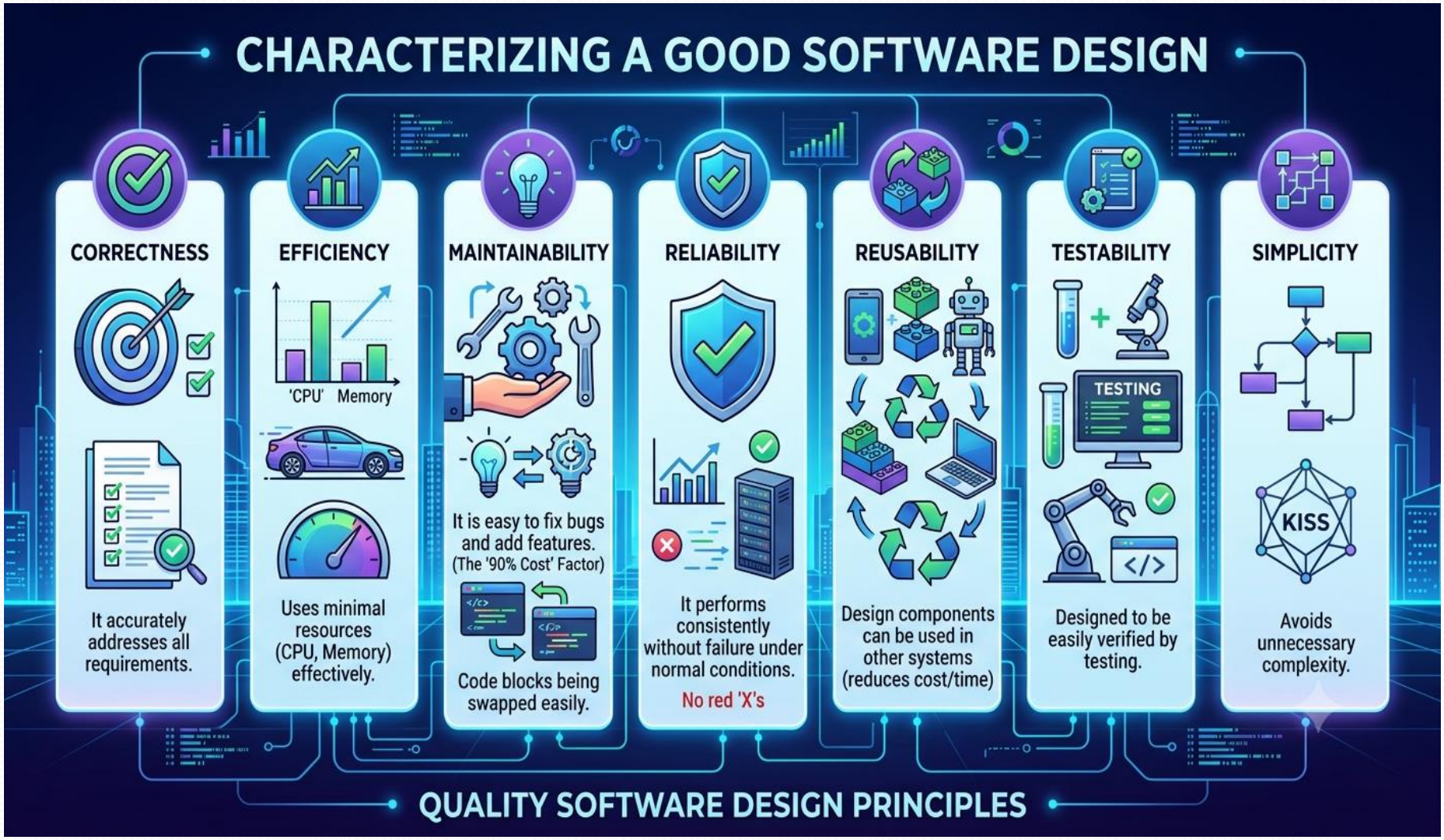
Overview of Design Process



Characteristics of Good Software Design

- **Correctness:** It accurately addresses all requirements.
- **Efficiency:** Uses minimal resources (CPU, Memory) effectively.
- **Maintainability:** It is easy to fix bugs and add features. (The "90% Cost" Factor).
- **Reliability:** It performs consistently without failure under normal conditions.
- **Reusability:** Design components can be used in other systems (reduces cost/time).
- **Testability:** Designed to be easily verified by testing.
- **Simplicity:** Avoids unnecessary complexity.

Characteristics of Good Software Design



Concept of Modularity

- **Definition:** Dividing the software into smaller, manageable, and independent components called "Modules."
- **Why Modularity?** It allows for parallel development, easier debugging, and better maintenance.
- **Two Key Principles:**
  - **High Cohesion (Inside the Module):** Elements within a module belong together.
  - **Low Coupling (Between Modules):** Modules should communicate as little as possible with each other.
-

COHESION IN SOFTWARE ENGINEERING

Cohesion measures how closely the elements inside a module belong together.

High cohesion → Better maintainability, readability and reusability.

LOW COHESION

HIGH COHESION

COINCIDENTAL COHESION

Unrelated tasks are grouped together in a module.

Avoid: Totally unrelated responsibilities.

```
class Utils {
    void printReport() { ... }
    void calculateSalary() { ... }
    void sendEmail() { ... }
    void backupDatabase() { ... }
}
```



Unrelated functions with no common purpose.

LOGICAL COHESION

Similar type of tasks are grouped together.

Better, but not ideal.

```
class DataUtils {
    void readFile() { ... }
    void writeFile() { ... }
    void deleteFile() { ... }
}
```



All operations related to files (same kind of activity).

TEMPORAL COHESION

Tasks are grouped because they are performed at the same time.
Better, but context-dependent.

```
class Startup {
    void openConnections() { ... }
    void loadConfigurations() { ... }
    void initializeModules() { ... }
}
```



All tasks executed during system startup.

FUNCTIONAL COHESION

All tasks contribute to one well-defined function.

Good cohesion.

```
class Invoice {
    void calculateTotal() { ... }
    void applyDiscount() { ... }
    void calculateTax() { ... }
    double getAmount() { ... }
}
```



All functions work together to calculate an invoice amount.

INFORMATIONAL COHESION

All tasks operate on the same data or information.

Best cohesion.

```
class EmployeeRepository {
    void addEmployee(Employee e) { ... }
    void updateEmployee(Employee e) { ... }
    void deleteEmployee(int id) { ... }
    Employee getEmployee(int id) { ... }
}
```



All functions operate on Employee data.

KEY TAKEAWAY

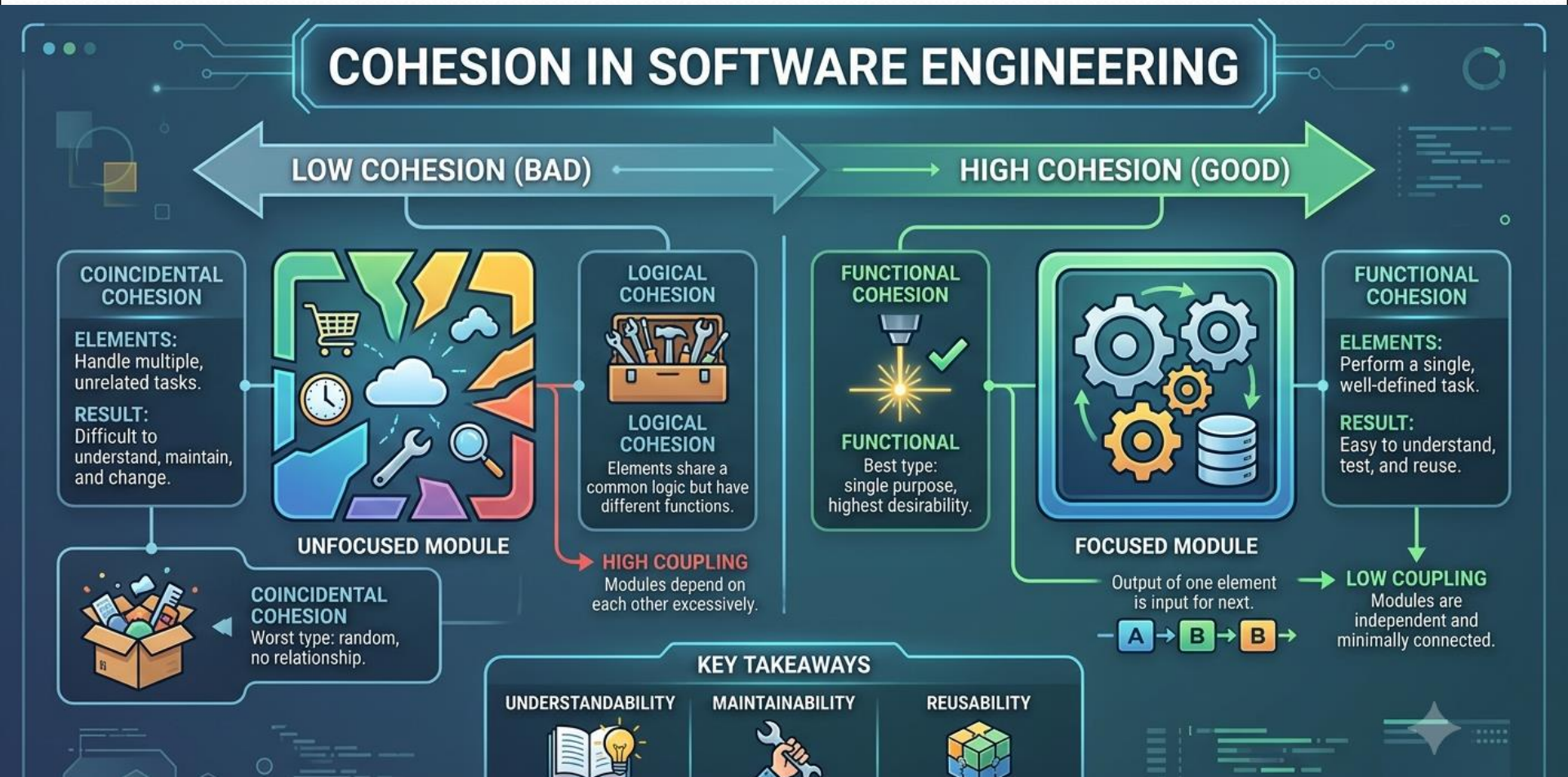
Strive for the highest possible cohesion. It leads to modules that are easier to understand, test, maintain and reuse.

TIP

As a rule of thumb, a module/class should have ONE reason to change.

Cohesion (Internal Strength)

- **Definition:** The degree to which the elements inside a single module belong together. It is a measure of the "single-mindedness" of a module.



Levels of Cohesion

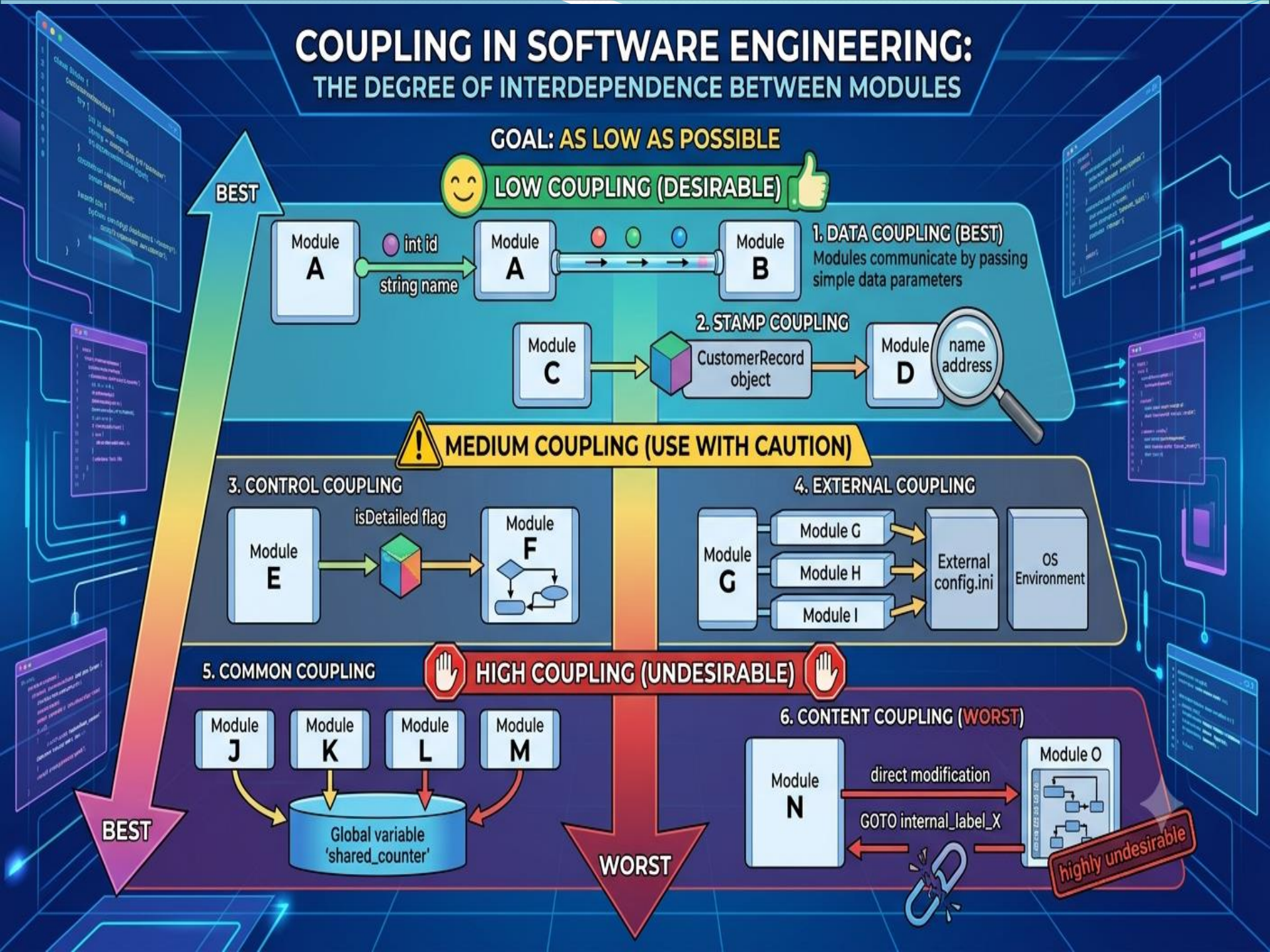
Functional (Best): The module performs exactly one specific task

- **Sequential:** The output of one part is the input to the next part.
- **Communicational:** Operates on the same data structure.
- **Procedural:** Parts are related by a sequence of steps.
- **Temporal:** Functions related in time (e.g., "Initialize").
- **Logical:** Similar tasks grouped (e.g., "Print" for different types).
- **Coincidental (Worst):** Random elements grouped together (e.g., "Utilities").

Coupling (Interdependence)

- **Definition:** The degree of interdependence between different modules. We want this to be *as low as possible*.

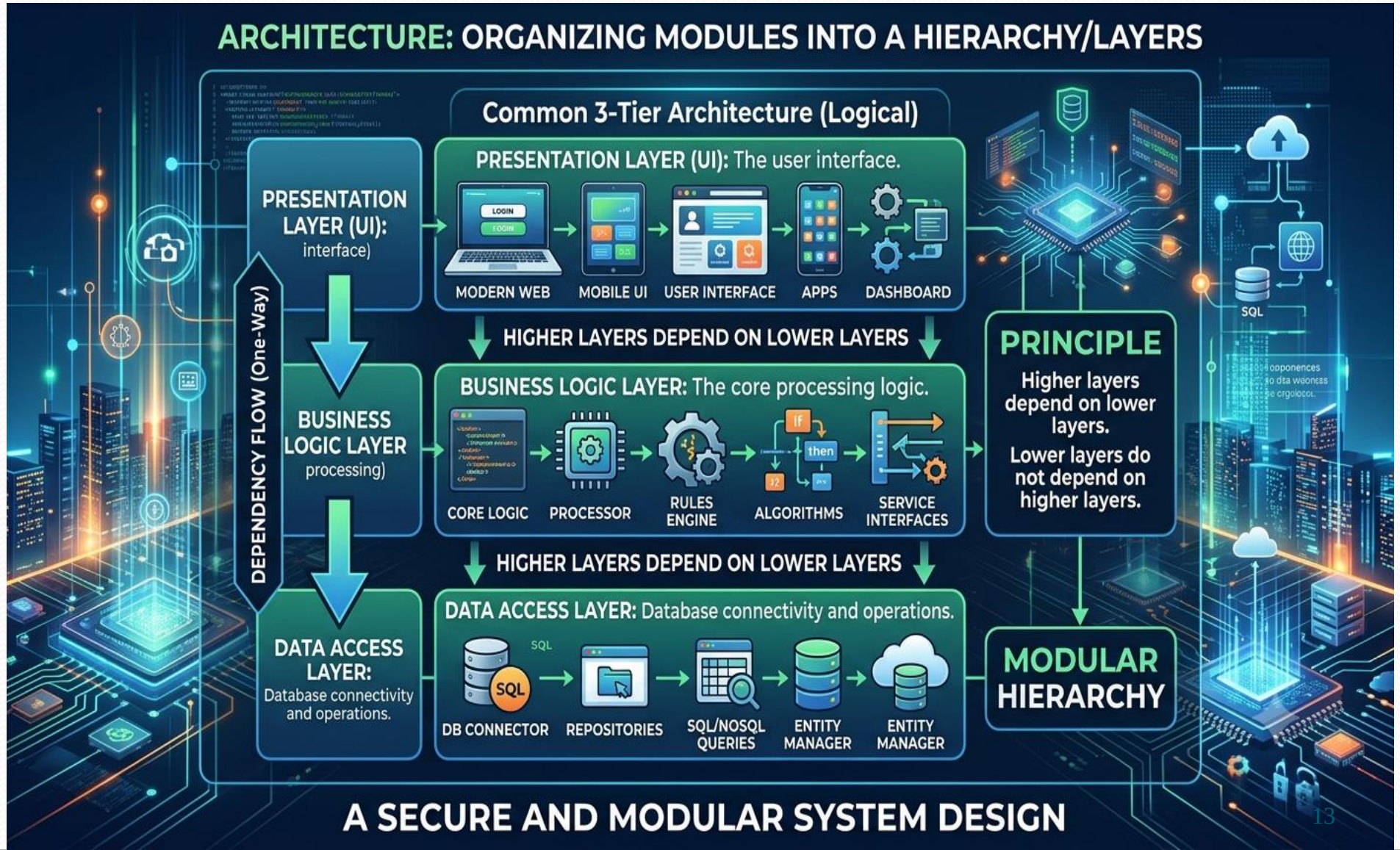
- **Levels of Coupling (Best to Worst):**
  - **Data (Best):** Modules communicate by passing simple data parameters.
  - **Stamp:** Modules pass data structures (objects/records), not just simple variables.
  - **Control:** One module controls the logic of another by passing flags/control parameters.
  - **External:** Modules communicate via a shared external environment (globals).
  - **Common:** Modules share global data.
  - **Content (Worst):** One module modifies the internal logic of another (e.g., GOTO statements). **This is highly undesirable.**


COUPLING IN SOFTWARE ENGINEERING:
THE DEGREE OF INTERDEPENDENCE BETWEEN MODULES
GOAL: AS LOW AS POSSIBLE
BEST
LOW COUPLING (DESIRABLE)
Module A
int id
string name
Module A
Module B
1. DATA COUPLING (BEST)
Modules communicate by passing simple data parameters
2. STAMP COUPLING
Module C
CustomerRecord object
Module D
name address
MEDIUM COUPLING (USE WITH CAUTION)
3. CONTROL COUPLING
Module E
isDetailed flag
Module F
4. EXTERNAL COUPLING
Module G
Module G
Module H
Module I
External config.ini
OS Environment
5. COMMON COUPLING
HIGH COUPLING (UNDESIRABLE)
Module J
Module K
Module L
Module M
Global variable 'shared_counter'
6. CONTENT COUPLING (WORST)
Module N
direct modification
GOTO internal_label_X
Module O
highly undesirable
BEST
WORST

Layered Arrangement of Modules

- **Architecture:** Organizing modules into a hierarchy/layers.
- **Principle:** Higher layers depend on lower layers. Lower layers do not depend on higher layers.
- **Common 3-Tier Architecture (Logical):**
  - **Presentation Layer (UI):** The user interface.
  - **Business Logic Layer:** The core processing logic.
  - **Data Access Layer:** Database connectivity and operations.
- **Benefits:**
  - **Separation of Concerns:** Each layer has a specific job.
  - **Maintainability:** Changes in UI don't affect the Database.
  - **Scalability:** Can scale specific layers independently.

Layered Arrangement of Modules



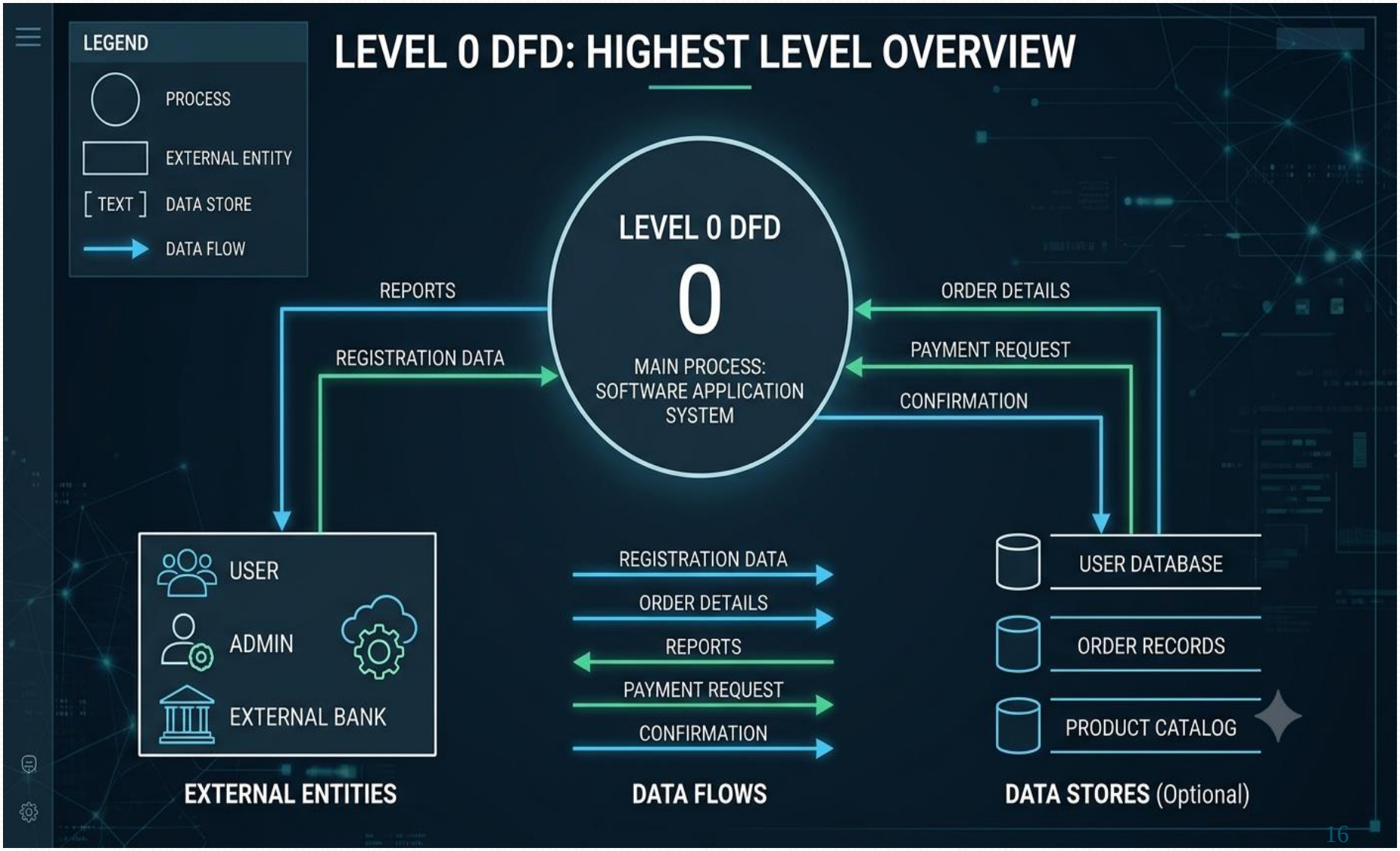
Function Oriented Design & DFD

- **Approach:** The focus is on the "function" or process.
- **Primary Tool:** Data Flow Diagram (DFD).
- **What is a DFD?** A graphical tool to show the flow of data through a system.
- **Components:**
  - **External Entity:** Source or destination of data (e.g., Customer).
  - **Process:** Transforms data (e.g., Calculate Order Total).
  - **Data Store:** Repository of data (e.g., Database).
  - **Data Flow:** The movement of data (Arrow).
- **Levels:**
  - **Level 0 (Context Diagram):** High-level view showing system boundaries.
  - **Level 1:** Decomposing main processes into sub-processes.
  - **Level 2:** Detailed view of internal processes.

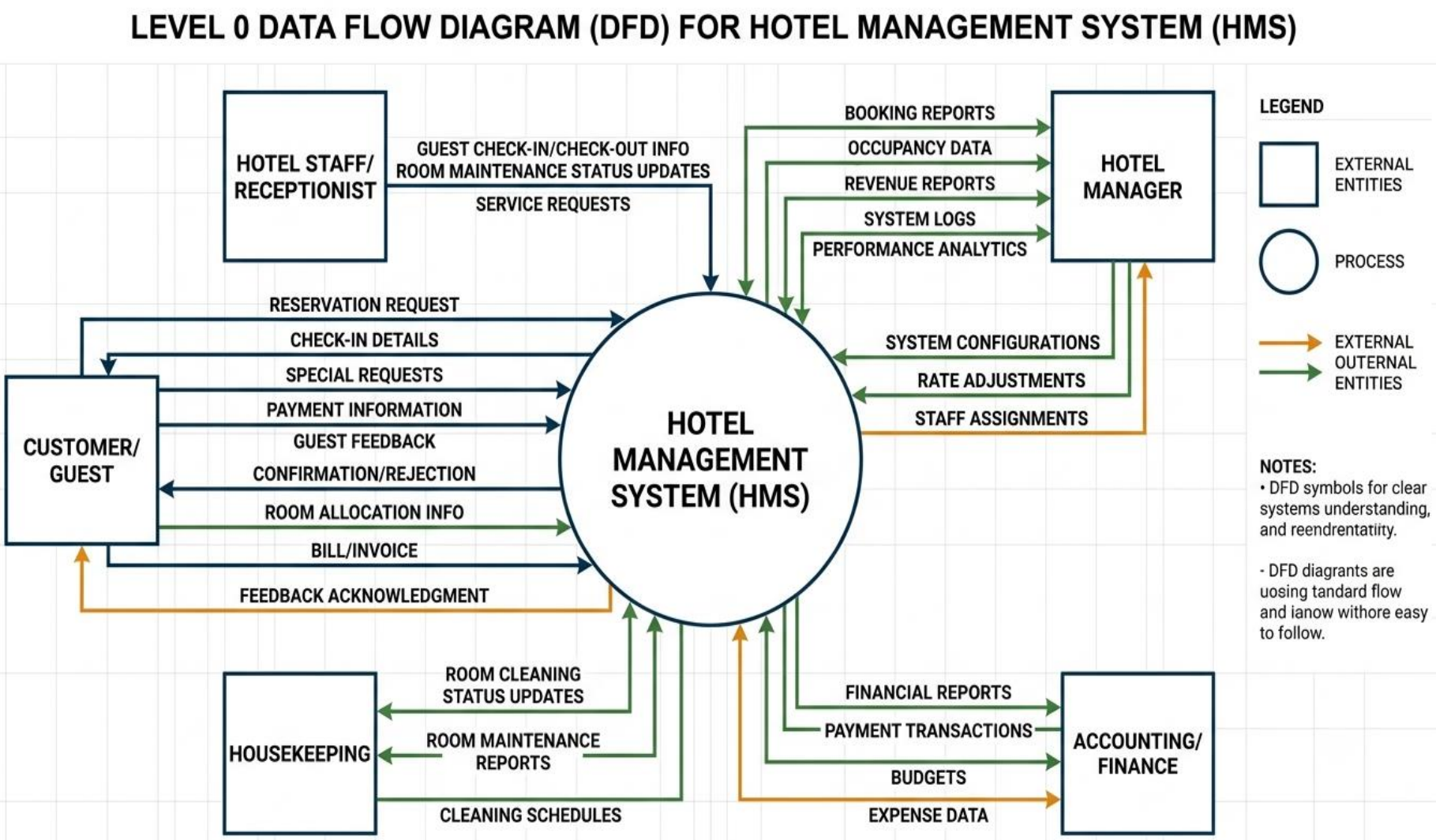
DFD (LEVEL 0)

- A Level o Data Flow Diagram (DFD), often called a Context Diagram, provides a high-level overview of an entire system.
- It acts as a single process block, illustrating how the system interacts with external entities by showing the fundamental data flows in and out.
- This visualization effectively captures the system's scope and its main interfaces with users, other systems, and databases without delving into internal process details.

DFD (LEVEL 0)


LEGEND
PROCESS
EXTERNAL ENTITY
[TEXT] DATA STORE
DATA FLOW
LEVEL 0 DFD: HIGHEST LEVEL OVERVIEW
LEVEL 0 DFD
0
MAIN PROCESS: SOFTWARE APPLICATION SYSTEM
REPORTS
REGISTRATION DATA
ORDER DETAILS
PAYMENT REQUEST
CONFIRMATION
USER
ADMIN
EXTERNAL BANK
EXTERNAL ENTITIES
REGISTRATION DATA
ORDER DETAILS
REPORTS
PAYMENT REQUEST
CONFIRMATION
DATA FLOWS
USER DATABASE
ORDER RECORDS
PRODUCT CATALOG
DATA STORES (Optional)

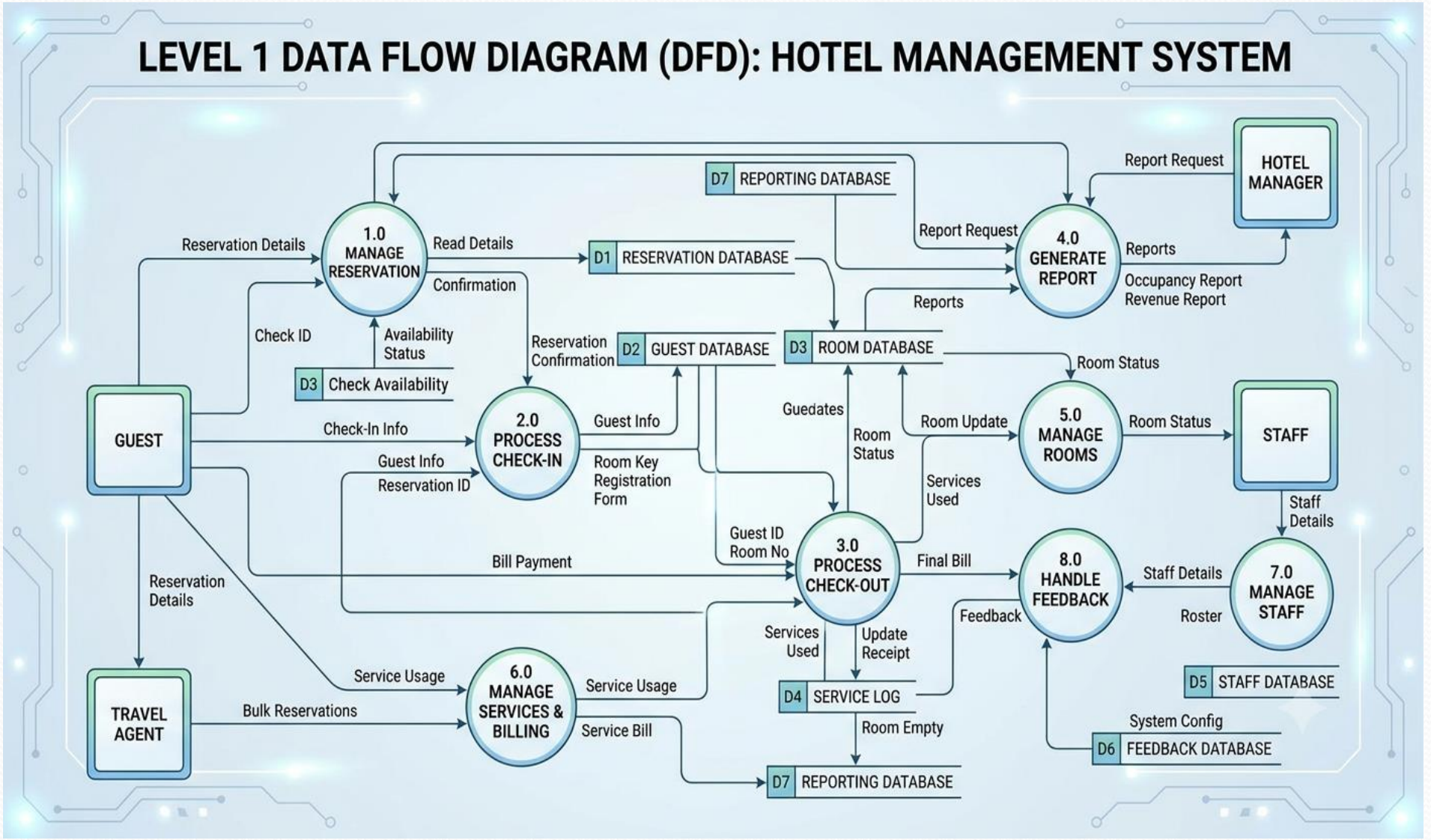
DFD (LEVEL 0) – Hotel Management System



DFD (LEVEL 1)

- A **Level 1 Data Flow Diagram (DFD)** takes the single core process from a Context Diagram (Level 0) and breaks it down into its major sub-processes. It bridges the gap between a high-level system overview and deep technical design, showing how information moves through the core functions, where it is stored, and who interacts with it.

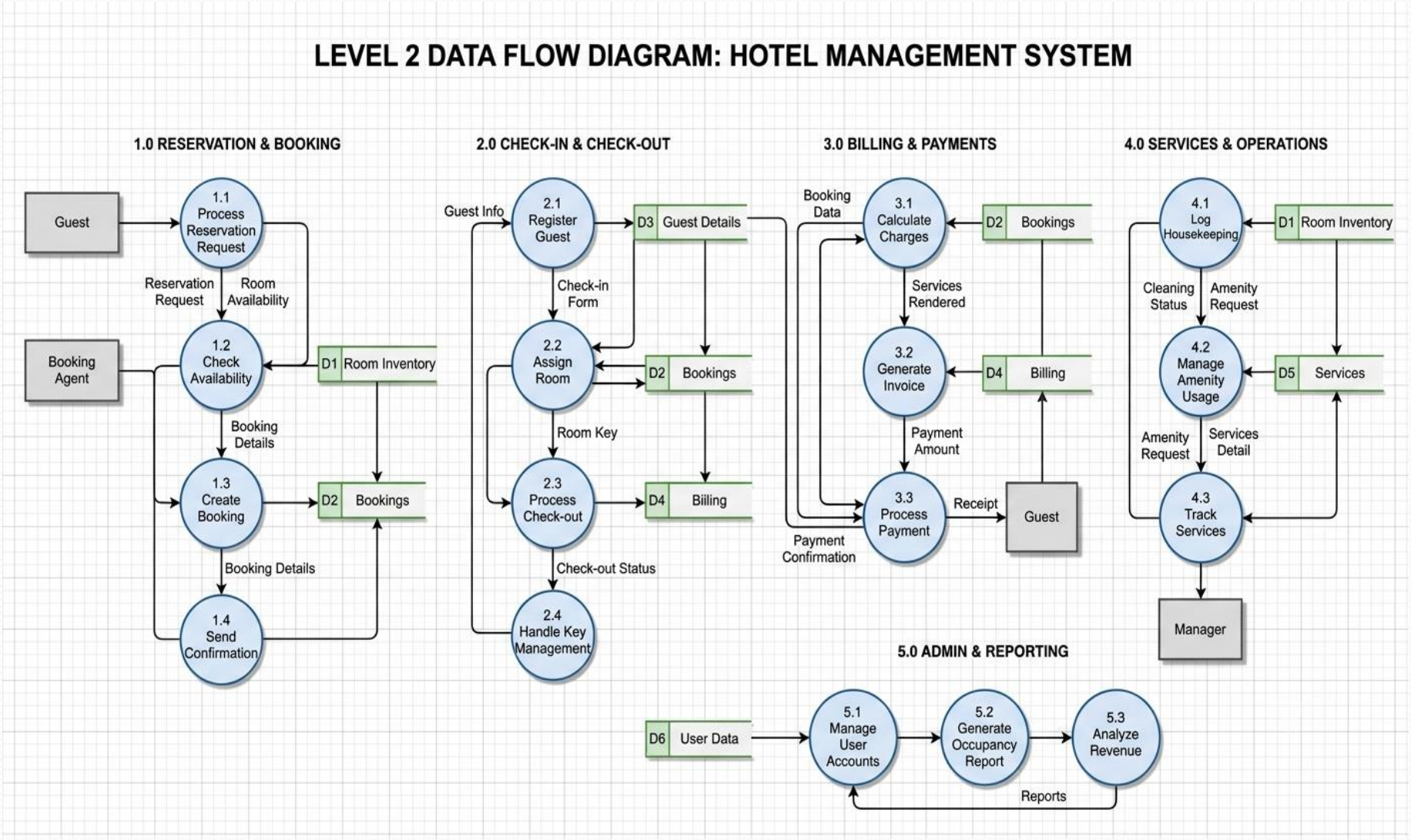
DFD (LEVEL 1) – Hotel Management System



DFD (LEVEL 2)

- A **Level 2 Data Flow Diagram (DFD)** is a highly detailed visual representation that breaks down a specific process from a Level 1 DFD into its smaller, more granular sub-processes.

DFD (LEVEL 2) – Hotel Management System



Introduction to Object Modeling Using UML

- **The Shift:** Moving from "Functions" to "Objects."
- **What is UML?** Unified Modeling Language. A standardized visual language for representing software architecture.
- **Why UML?** Facilitates communication among stakeholders (Analysts, Designers, Coders).
- **Two Main Categories of Diagrams:**
  - **Structural Diagrams:** Represent the static structure.
  - **Behavioral Diagrams:** Represent the dynamic behavior.
- *Note:* We will focus on Interaction, State Chart, and the concepts of Class Relationships.

Main Categories of UML Diagram

•**Structural Diagrams:** This section includes Class Diagrams (attributes/relationships), Component Diagrams (interconnected modules), Deployment Diagrams

• (hardware/software distribution), Package Diagrams (organizing elements), and Composite Structure Diagrams (internal parts).

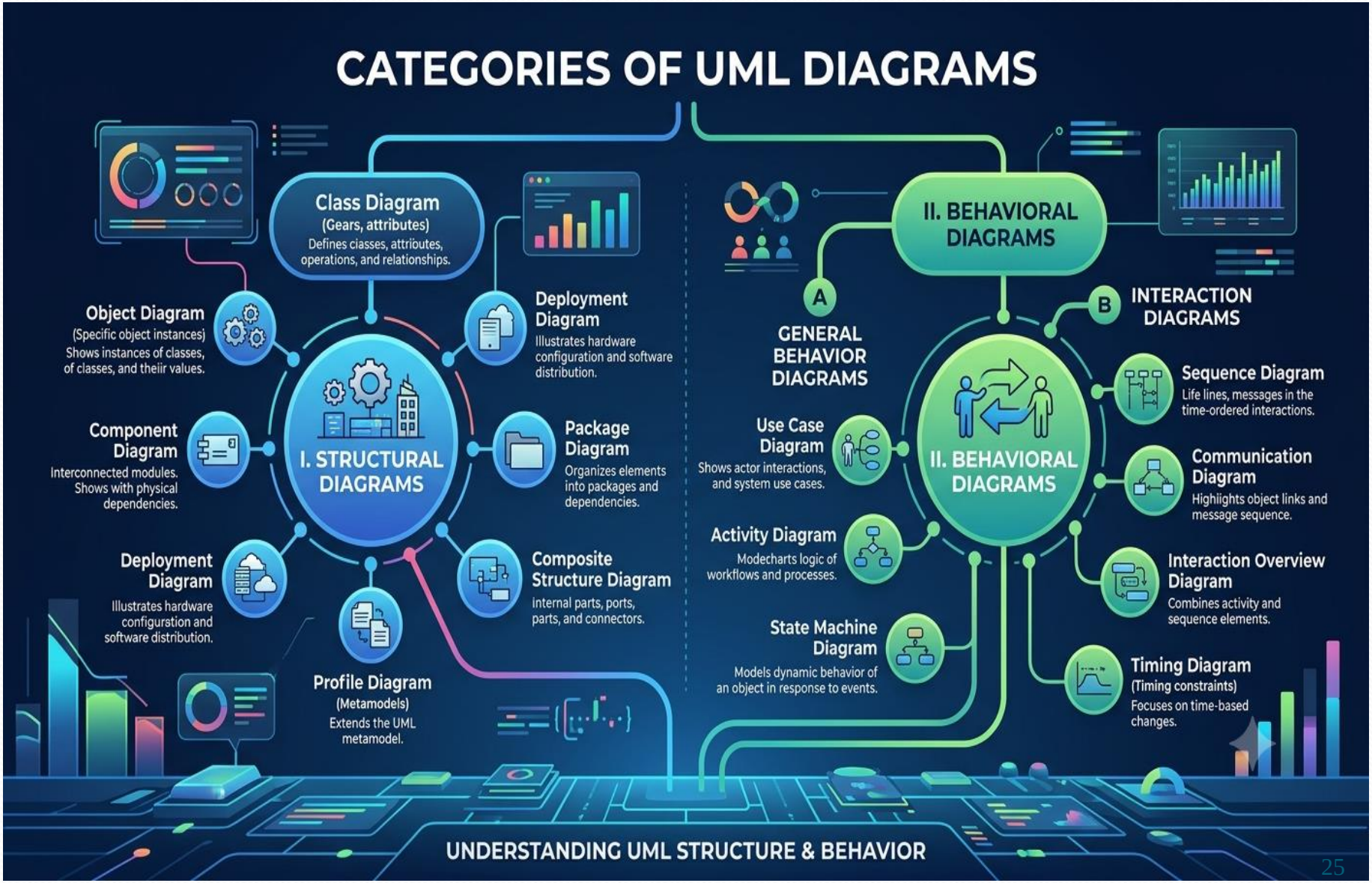
Main Categories of UML Diagram

•**Behavioral Diagrams:** This category is further divided:

A. General Behavior Diagrams: Including Use Case Diagrams (actor interactions), Activity Diagrams (workflows/processes), and State Machine Diagrams (object states).

B. Interaction Diagrams: A subset of Behavioral Diagrams that includes Sequence Diagrams (time-ordered messages), Communication Diagrams (object links), Interaction Overview Diagrams (workflows with interaction elements), and Timing Diagrams (timing constraints).

Main Categories of UML Diagram



- REST PART IE. USE CASE , CLASS , ACTIVITY , SEQUENCE DIAGRAM HAS ALREADY COVERED IN UNIT II.